

传感器新一代安全通信协议落地 Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: 将《传感器新一代安全通信协议卷》落地为 CPU2、传感器端、测试工具和 SIL 证据可同步推进的实施方案。

Architecture: 采用分层协议栈：字节流传输层只负责 UART6/RS485 收发，帧层负责 CRC32C、长度、字节序和字段偏移，会话层负责 `session_id`、`seq`、`safety_param_crc` 和 `config_epoch`，业务适配层只输出密度、温度、频率、状态和诊断快照。周期上报使用固定 44 字节 `FAST_REPORT_FRAME`，控制命令使用 `CTRL_FRAME`。

Tech Stack: C11、STM32 HAL UART/DMA、RS485 9600 bps、Python 3 验证脚本、Markdown 认证证据。

1. 落地边界

本方案只规划新协议落地，不直接修改源码。执行时按任务逐步实现，任何一步完成后都要能单独构建、测试和回退。

正式协议源文件：

- `docs/01_协议与寄存器/传感器通信协议/传感器新一代安全通信协议卷.md`

本方案覆盖范围：

- CPU2 新协议栈实现。
- 传感器端最小实现要求。
- 9600 bps、200 米 RS485 联调和实时性验证。
- golden frame、故障注入和长稳测试。
- SIL 认证资料需要同步保留的证据。

本方案不覆盖：

- CPU3、Modbus、SI7000 或外部协议的寄存器兼容改造。只有当新传感器状态需要暴露给 CPU3 或外部系统时，才进入对应协议目录和 `DEVICE_PROTOCOL_VERSION` 评估。
- 协议内密钥认证。主动攻击、无线开放接入或非受控接入风险由系统安全方案补充。
- PDF 同步。Markdown 是权威正文，PDF 不作为实施阻塞项。

2. 总体实施顺序

实施顺序必须先测试工具、再纯协议栈、再 UART 接入、再业务接入、最后硬件联调。这样可以避免把 CRC、字节序、状态机和业务测量问题混在一起。

阶段	目标	主要输出
P0 协议冻结	固定字段、命令、错误码和验收口径	协议卷复核记录、字段冻结清单
P1 golden frame	先生成跨端一致的标准帧	JSON 向量、Python 校验脚本

阶段	目标	主要输出
P2 CPU2 纯协议栈	不依赖 UART 的编解码和校验	<code>sensor_safe_*</code> 帧层和会话层
P3 CPU2 UART6 传输	接入 9600 bps RS485 半双工收发	UART6 transport、超时和收发窗口
P4 CPU2 业务适配	提供当前测量流程可调用的新服务接口	新协议 client、快报缓存、双向读数
P5 传感器端实现	传感器按最小状态机实现协议	控制帧命令、44 字节快报、退出上报
P6 联调验证	覆盖正常、异常、长稳和 200 米链路	测试记录、波形/日志、故障注入结果
P7 认证资料	形成可追溯证据包	需求追踪、验证矩阵、残余风险说明

3. CPU2 文件结构建议

新增文件建议全部放在 `LTD_MAIN_CPU2/Services/Sensor/`。当前 `LTD_MAIN_CPU2/CMakeLists.txt` 使用 `file(GLOB_RECURSE SERVICE_SOURCES ... Services/*.c)`，新增 `.c` 文件会自动进入 CMake 构建；STM32CubeIDE 工程也已包含 `Services/Sensor` 目录，执行时仍要确认 IDE 没有排除新文件。

文件	职责
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_types.h</code>	协议常量、命令枚举、状态位、错误码、字段偏移、固定长度
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_crc32c.h</code>	CRC32C API 声明
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_crc32c.c</code>	CRC32C Castagnoli 软件实现和测试向量入口
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_frame.h</code>	控制帧和快报帧编解码 API
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_frame.c</code>	小端读写、控制帧封包、控制帧解包、快报帧解包
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_session.h</code>	CPU2 会话、事务、上报流状态结构
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_session.c</code>	<code>session_id</code> 、 <code>seq</code> 、 <code>safety_param_crc</code> 、 <code>config_epoch</code> 、 <code>stream_seq</code> 校验
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_transport_uart6.h</code>	UART6/RS485 传输接口
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_transport_uart6.c</code>	半双工发送、DMA 接收、快报帧扫描、超时处理
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_client.h</code>	CPU2 上层可调用的新协议服务 API
<code>LTD_MAIN_CPU2/Services/Sensor/sensor_safe_client.c</code>	<code>HELLO</code> 、 <code>SET_MEASURE_MODE</code> 、 <code>READ_MEAS_COMBO</code> 、 <code>SET_COMM_MODE</code> 、 <code>GET_STATUS</code> 等命令

文件	职责
LTD_MAIN_CPU2/Services/Sensor/sensor_safe_service.h	当前 <code>sensor.h</code> 业务接口的适配层
LTD_MAIN_CPU2/Services/Sensor/sensor_safe_service.c	将新协议测量结果转换成当前测量流程需要的 <code>float</code> 或 RAW 值

执行原则：

- 新协议栈不改旧 `dsm_sensor_communication.c` 和 `ltd_sensor_communication.c` 的协议逻辑。
- 初期通过编译开关或运行时传感器类型选择新旧协议，便于回退。
- 业务层可以先保留 `Read_Density()` 等旧接口作为桥接，CPU2 上层重做后再直接调用 `SensorSafeClient_*`。
- 旧源码可能是 GBK/936，修改 `.c/.h` 前必须确认编码；新增文件建议使用 UTF-8，但不得把旧文件无意转码。
- C/C++ 注释统一使用 `/* ... */`，不得新增 `//` 注释。

4. 工具和测试文件建议

文件	职责
tools/sensor_safe_protocol_vectors.py	生成 golden frame 和 CRC32C 测试向量
tools/check_sensor_safe_protocol_contract.py	校验协议卷、golden frame、字段长度和 CRC 计算范围一致
docs/01_协议与寄存器/传感器通信协议/golden_frames/sensor_safe_protocol_v1.json	固定 golden frame 数据
docs/01_协议与寄存器/传感器通信协议/传感器新一代安全通信协议验证记录.md	记录 PC 仿真、硬件联调、长稳和故障注入结果

建议先做 Python golden frame，再做 CPU2 C 代码。Python 工具必须输出以下帧：

- HELLO 请求和响应。
- GET_CONFIG_DIGEST 响应。
- SET_MEASURE_MODE(DENSITY) 请求和响应。
- READ_MEAS_COMBO 请求和响应。
- SET_COMM_MODE(PERIODIC_UPLINK) 请求和响应。
- REPORT_MEAS_FAST 连续 3 帧。
- SET_COMM_MODE(REQUEST_RESPONSE) 请求和响应。
- CRC 错、长度错、地址错、会话错、`config_epoch` 错、`stream_seq` 重复和跳变帧。

工具验证命令：

```
py tools\sensor_safe_protocol_vectors.py --write docs\01_协议与寄存器\传感器通信协议\golden_frames\sensor_safe_protocol_v1.json
```

```
py tools\check_sensor_safe_protocol_contract.py
py tools\check_docs.py
```

通过标准：

- golden frame 中每帧 CRC32C 可重复计算一致。
- `FAST_REPORT_FRAME` 固定 44 字节。
- 控制帧 `frame_len <= 128`。
- 所有多字节字段按小端序编码。
- 协议卷字段偏移和 JSON 向量字段偏移一致。

5. CPU2 详细任务

Task 1: 协议常量和类型冻结

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_types.h`
- Read: `docs/01_协议与寄存器/传感器通信协议/传感器新一代安全通信协议卷.md`
- ☐ Step 1: 定义协议版本、节点默认 ID、控制帧最大长度、快报固定长度、SOF、`frame_type=0xA1`。
- ☐ Step 2: 定义 `cmd`、`msg_type`、测量模式、通信模式、状态位、诊断位、错误码枚举。
- ☐ Step 3: 定义 `FAST_REPORT_FRAME` 每个字段的偏移宏，偏移必须与协议卷第 5.2 节一致。
- ☐ Step 4: 定义 CPU2 侧测量快照结构，字段使用整数倍率保存，再由适配层转换给旧业务。
- ☐ Step 5: 编译 CPU2，确保头文件无循环包含。

验收命令：

```
cmake --build build\LTD_MAIN_CPU2
```

验收标准：

- 编译通过。
- `sensor_safe_types.h` 不包含 HAL 头文件。
- 快报偏移宏最后满足 `SENSOR_SAFE_FAST_FRAME_LEN == 44U`。

Task 2: CRC32C 实现

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_crc32c.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_crc32c.c`
- Create or update: `tools/check_sensor_safe_protocol_contract.py`
- ☐ Step 1: 先在 Python 工具中固定 CRC32C 测试向量。

- ☐ Step 2: C 端实现 `SensorSafe_Crc32c(const uint8_t *data, uint16_t len)`。
- ☐ Step 3: 在工具中导出同一输入的期望 CRC。
- ☐ Step 4: 用 CPU2 构建确认没有引入库依赖。
- ☐ Step 5: 后续硬件联调时打印或断点比对 C 端 CRC 与 Python 向量。

验收标准：

- 空负载、1 字节、44 字节快报、128 字节控制帧边界均有向量。
- CRC 初值、输入反射、输出异或和字节序在工具和 C 端完全一致。
- 文档中明确 CRC 是错误检测码，不作为认证码。

Task 3: 控制帧编解码

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_frame.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_frame.c`
- ☐ Step 1: 实现小端 `read_u16/read_u32/write_u16/write_u32` 内部函数。
- ☐ Step 2: 实现 `CTRL_FRAME` 封包，输入为头字段和 payload。
- ☐ Step 3: 实现 `CTRL_FRAME` 解包，校验 SOF、长度、`header_len`、版本、CRC、地址。
- ☐ Step 4: 对 `HELLO` 允许 `session_id=0` 和 `safety_param_crc=0`，其它命令必须由会话层检查。
- ☐ Step 5: 禁止结构体强转，所有字段按偏移逐字节读写。

验收标准：

- 可生成与 Python golden frame 完全一致的 `HELLO`、`READ_MEAS_COMBO`、`SET_COMM_MODE` 请求帧。
- 短帧、超长、CRC 错、地址错、版本错均被拒绝。
- 解包函数不访问 payload 边界外数据。

Task 4: 快报帧解码和数据快照

Files:

- Modify: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_frame.h`
- Modify: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_frame.c`
- ☐ Step 1: 实现 `FAST_REPORT_FRAME` 解码，固定检查 `0xA5 0x5A`、`frame_type=0xA1`、`frame_len=44`。
- ☐ Step 2: 校验快报 CRC32C 覆盖 `frame_type` 到 `fast_reserved`。
- ☐ Step 3: 按偏移读取 `src_id`、`dst_id`、`session_id`、`stream_id`、`stream_seq`、`sample_counter`、`data_age_ms`、状态位、诊断位和测量值。
- ☐ Step 4: 输出整数倍率快照，不在帧层转 `float`。

- ☐ Step 5: 对 `stream_seq` 和 `sample_counter` 只输出原始 u16，连续性由会话层判断。

验收标准：

- 正常 3 帧 golden frame 全部解码成功。
- 任意单 bit 错误导致 CRC 失败。
- `src_id` 或 `dst_id` 错误不进入业务层。

Task 5: 会话和上报流状态机

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_session.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_session.c`
- ☐ Step 1: 定义 CPU2 会话状态：`OFFLINE`、`SESSION_READY`、`MEASURE_READY`、`PERIODIC_ACTIVE`、`RECOVERING`、`FAULT`。
- ☐ Step 2: 保存当前 `session_id`、`sensor_id`、`safety_param_crc`、`scale_version`、`config_epoch`、`stream_id`、最近 seq、最近 `stream_seq`。
- ☐ Step 3: 实现双向响应校验：地址、会话、`seq`、`cmd`、`safety_param_crc`、`result_code`、状态位、`data_age_ms`。
- ☐ Step 4: 实现快报校验：地址、会话、流 ID、`stream_seq` 模 65536 连续性、`sample_counter` 冻结判断、`config_epoch`、状态位、`data_age_ms`。
- ☐ Step 5: 实现配置变化处理：`config_epoch` 不一致时丢弃快报并要求重新 `GET_CONFIG_DIGEST` 或 `HELLO`。

验收标准：

- 重复快报、跳号快报、旧会话快报、旧流快报均不会更新安全相关测量值。
- `stream_seq` 从 65535 到 0 的合法回绕可识别。
- 同一 `session_id` 内 `config_epoch` 回绕复用被拒绝。

Task 6: UART6/RS485 传输层接入

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_transport_uart6.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_transport_uart6.c`
- Read: `LTD_MAIN_CPU2/Services/Sensor/dsm_sensor_communication.c`
- Read: `LTD_MAIN_CPU2/Core/Src/usart.c`
- ☐ Step 1: 复用现有 UART6 DMA 收发经验，但不要调用 DSM 文本协议的 BCC 或换行等待逻辑。
- ☐ Step 2: 实现控制帧发送前清空旧接收状态，发送后按 `frame_len` 或固定头部长度的接收响应。
- ☐ Step 3: 实现快报帧扫描状态机：查找 SOF、固定读取 44 字节、校验后交给会话层。

- ☐ Step 4: 周期上报期间保留下行控制能力，退出上报命令必须能在 `control_guard_ms` 窗口发送。
- ☐ Step 5: 记录 UART 硬件错误、超时、短帧、CRC 错、地址错的计数，后续映射到诊断日志。

验收标准：

- 不依赖换行符判断帧边界。
- 9600 bps 下 `REPORT_MEAS_FAST` 收帧不阻塞主测量流程。
- 响应超时按协议卷 9.3 节预算处理。
- 任一 UART 错误不会把旧缓存数据当作新数据。

Task 7: CPU2 高层 client

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_client.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_client.c`
- ☐ Step 1: 实现 `SensorSafeClient_Init()`，初始化节点 ID、超时、会话状态和统计计数。
- ☐ Step 2: 实现 `SensorSafeClient_Hello()`，保存 `session_id`、`sensor_id`、`scale_version`、`safety_param_crc`。
- ☐ Step 3: 实现 `SensorSafeClient_SetMeasureMode()`。
- ☐ Step 4: 实现 `SensorSafeClient_ReadMeasureCombo()`。
- ☐ Step 5: 实现 `SensorSafeClient_EnterPeriodic()` 和 `SensorSafeClient_ExitPeriodic()`。
- ☐ Step 6: 实现 `SensorSafeClient_PollFastReport()`，只在会话层校验通过后更新最新快照。
- ☐ Step 7: 实现 `SensorSafeClient_GetStatus()` 和 `SensorSafeClient_GetCounters()`，便于调试和认证记录。

验收标准：

- 每次只允许一个未完成请求。
- 有副作用命令使用 `seq` 做幂等保护。
- 退出周期上报成功后，同一 `session_id` 下可继续双向读取。

Task 8: 当前业务适配

Files:

- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_service.h`
- Create: `LTD_MAIN_CPU2/Services/Sensor/sensor_safe_service.c`
- Modify with care: `LTD_MAIN_CPU2/Services/Sensor/sensor.c`
- Modify with care: `LTD_MAIN_CPU2/Services/Sensor/sensor.h`
- Read: `LTD_MAIN_CPU2/Application/Src/measure_density.c`
- ☐ Step 1: 定义新协议启用条件，例如编译开关或传感器类型识别结果。

- ☐ Step 2: 先让 `Read_Density()` 在新协议启用时调用 `SensorSafeClient_ReadMeasureCombo()`，输出当前业务仍需要的 `float frequency/density/temp`。
- ☐ Step 3: 保持旧 DSM/LTD 协议路径可回退。
- ☐ Step 4: 对周期上报建立最新快照缓存，业务层读取时必须检查快照有效性、`data_age_ms` 和诊断位。
- ☐ Step 5: 新增错误映射，把 CRC 错、静默超时、配置不一致、数据过期、诊断异常映射到现有错误日志体系。

验收标准：

- 不改变未启用新协议时的旧传感器行为。
- 新协议读数输出和协议整数倍率一致。
- 数据无效时不能沿用上一帧安全相关测量值。

6. 传感器端详细任务

传感器端固件不在当前仓库时，按本节模块名在传感器项目中建立同等职责文件。传感器端优先实现最小集合，不引入动态内存、分片、多事务或多上报流。

建议文件：

- `sensor_safe_types.h`
- `sensor_safe_crc32c.c/h`
- `sensor_safe_frame.c/h`
- `sensor_safe_session.c/h`
- `sensor_safe_measure.c/h`
- `sensor_safe_stream.c/h`
- `sensor_safe_uart.c/h`

传感器端任务：

- ☐ 实现控制帧接收状态机：SOF、长度、最大 128 字节、CRC32C、地址、版本、`header_len`。
- ☐ 实现 HELLO，分配新的 `session_id`，返回能力位、`scale_version`、`safety_param_crc`。
- ☐ 实现 GET_CONFIG_DIGEST，返回当前倍率、标定、参数表版本和配置 CRC。
- ☐ 实现 SET_MEASURE_MODE，模式切换期间置 `MODE_SETTLING`，稳定后置 `MODE_MATCH`。
- ☐ 实现 READ_MEAS_COMBO，从同一次采样快照返回温度、密度、频率和质量值。
- ☐ 实现 SET_COMM_MODE(PERIODIC_UPLINK)，校验 `report_cmd=REPORT_MEAS_FAST`、`reserved0=0`、周期、静默时间和 `stream_id`。
- ☐ 实现固定 44 字节 FAST_REPORT_FRAME，每帧更新 `stream_seq`、`sample_counter`、`data_age_ms`、状态位和诊断位。
- ☐ 实现 SET_COMM_MODE(REQUEST_RESPONSE)，停止当前上报流并返回 ACK。
- ☐ 实现 GET_STATUS、PING、RESET_SESSION。
- ☐ 实现错误响应：BAD_CRC、BAD_LENGTH、BAD_ARGUMENT、BAD_STATE、UNSUPPORTED_CMD、CONFIG_MISMATCH。

传感器端验收标准：

- 周期上报期间仍能接收退出命令。
- 配置变化时停止当前上报流或使 `config_epoch` 变化，CPU2 不会继续使用旧流数据。
- 快报帧只按固定 44 字节发送，不根据保留字段变长。

- 同一次快报中的温度、密度、频率来自同一测量快照或带有明确 `sample_counter`。

7. 联调和验证计划

7.1 PC 仿真先行

- ☐ PC 端模拟传感器，CPU2 或协议库向模拟器发送 `HELLO`、`READ_MEAS_COMBO`、`SET_COMM_MODE`。
- ☐ 模拟器按 `golden frame` 返回正常响应。
- ☐ 模拟器注入 CRC 错、短帧、乱序、重复帧、旧会话、错地址、`config_epoch` 错。
- ☐ CPU2 侧记录每种异常的拒绝原因。

通过标准：

- 所有异常帧不更新安全相关测量值。
- 所有正常帧能还原成协议卷定义的工程值。
- 错误计数和日志可追溯到故障类型。

7.2 板级短线联调

- ☐ CPU2 和传感器短线 9600 bps 通信。
- ☐ 完成上电 `HELLO`、设置密度模式、双向读取、进入周期上报、退出周期上报、继续双向读取。
- ☐ 抓取串口原始 HEX，与 `golden frame` 对比字段位置和 CRC。

通过标准：

- `READ_MEAS_COMBO` 请求和响应在普通请求超时预算内完成。
- `REPORT_MEAS_FAST` 周期稳定，1000 ms 周期下无长期漂移。
- 退出上报后不再接受旧 `stream_id` 数据。

7.3 200 米 RS485 长线验证

- ☐ 使用目标线缆、终端电阻、偏置电阻和实际接地方式搭建 200 米链路。
- ☐ 9600 bps 连续运行 24 小时，周期 1000 ms。
- ☐ 记录总帧数、CRC 错、短帧、静默超时、重试次数、退出上报成功率。
- ☐ 在不改变项目基线的前提下，可追加 19200 bps 和 38400 bps 探索性测试，结果只作为现场能力参考。

通过标准：

- 9600 bps 为正式通过项。
- `stream_seq` 至少覆盖一次 u16 回绕的仿真测试；硬件 1000 ms 长稳建议覆盖 24 小时。
- 任一通信异常都触发数据无效或恢复流程，不产生未标识的旧数据。

7.4 故障注入矩阵

类别	注入方式	期望结果
数据损坏	翻转任意 bit	CRC 错，丢弃
截断	少发 1 到 10 字节	长度或超时失败，丢弃
粘包	两帧连续无间隔	能按 SOF 和长度重新同步

类别	注入方式	期望结果
错地址	改 <code>src_id</code> 或 <code>dst_id</code>	丢弃，不更新数据
旧会话	使用旧 <code>session_id</code>	丢弃
重放	重复旧 <code>stream_seq</code>	丢弃或标记重复
丢帧	跳过一个 <code>stream_seq</code>	标记丢帧，按 SRS 策略处理
冻结	<code>sample_counter</code> 长期不变	数据无效
配置变化	改 <code>config_epoch</code>	丢弃并重新确认配置
数据过期	<code>data_age_ms</code> 超限	不作为安全相关输入
传感器故障	<code>DATA_VALID=0</code> 或诊断位置位	不更新安全相关测量值

8. SIL 证据同步要求

实现过程中必须同步积累下列证据，避免代码完成后再补认证资料。

证据	产物	对应标准主题
需求追踪	协议卷第 1 章和实现任务追踪表	IEC 61508-1 安全生命周期
架构分层	CPU2 文件结构和传感器端文件结构评审记录	IEC 61508-3 软件架构
接口明确性	字段偏移、类型、字节序、倍率表、golden frame	IEC 61508-3 接口规格
black channel 防护	CRC、地址、会话、序号、数据年龄、状态诊断验证记录	IEC 61784-3
通信失效处理	故障注入矩阵执行记录	IEC 62280
配置管理	<code>scale_version</code> 、 <code>safety_param_crc</code> 、 <code>config_epoch</code> 变化测试	IEC 61508-1 配置管理
残余风险	CRC32C 充分性、主动攻击边界、目标 SIL 来源说明	IEC 61508、IEC 62443

认证前必须完成：

- CRC32C 对最大帧长、误码率和目标危险失效率的充分性论证。
- 整机 SRS 给出最大允许数据年龄、最大静默时间和安全功能响应时间。
- 诊断覆盖率估算和故障注入结果闭环。
- 现场参数修改权限、审计、备份和回滚流程。

9. 版本、提交和流程文档影响

普通实现阶段不主动升 CPU2/CPU3 固件版本。提交或发布前按最终 diff 判断：

- 只新增协议文档和工具，不改变固件输出：通常不升级固件版本。
- CPU2 新增协议栈但未启用：评估 CPU2 固件版本，通常不改变 CPU2/CPU3 共享 `DEVICE_PROTOCOL_VERSION`。

- CPU2 默认启用新传感器协议：必须升级 CPU2 固件版本，并更新 [CHANGELOG.md](#) 和版本改动与测试方案。
- 新状态、诊断或测量值暴露到 CPU3/Modbus/SI7000：必须评估 [DEVICE_PROTOCOL_VERSION](#)，同步 CPU2/CPU3 协议文档和变更记录。
- 若最终 diff 改变传感器通信流程、命令分发、测量结果上报或异常恢复路径，必须同步 [docs/00_程序流程导航/CPU2/09_传感器与无线通信.html](#) 及对应业务流程图。

提交前建议命令：

```
py tools\check_sensor_safe_protocol_contract.py
py tools\check_docs.py
cmake --build build\LTD_MAIN_CPU2
py tools\check_commit_subject_versions.py --range origin/MAIN..HEAD
```

10. 执行切分建议

建议按以下小批次执行，每批次完成后单独评审。

Batch A: 协议工具和 golden frame

- ☐ 新增 Python golden frame 生成工具。
- ☐ 新增 golden frame JSON。
- ☐ 新增协议契约检查脚本。
- ☐ 运行 `py tools\check_sensor_safe_protocol_contract.py`。
- ☐ 运行 `py tools\check_docs.py`。

完成标准：不需要硬件即可证明协议卷、字段偏移、CRC 和 golden frame 一致。

Batch B: CPU2 纯协议栈

- ☐ 新增 `sensor_safe_types`。
- ☐ 新增 `sensor_safe_crc32c`。
- ☐ 新增 `sensor_safe_frame`。
- ☐ 新增 `sensor_safe_session`。
- ☐ 构建 CPU2。

完成标准：不接 UART，只用 golden frame 验证编解码和会话校验。

Batch C: CPU2 UART6 传输和 client

- ☐ 新增 `sensor_safe_transport_uart6`。
- ☐ 新增 `sensor_safe_client`。
- ☐ 对接 UART6 DMA 和超时。
- ☐ 完成 PC 模拟传感器联调。

完成标准：CPU2 可和 PC 模拟器完成完整握手、双向读数、进入上报、退出上报。

Batch D: CPU2 业务适配

- ☐ 新增 `sensor_safe_service`。

- ☐ 在 `sensor.c` 中增加新协议选择入口。
- ☐ 保留旧协议回退路径。
- ☐ 更新错误日志映射。

完成标准：当前 `SinglePoint_ReadSensor()` 路径可通过新协议拿到密度、温度、频率，异常时不会使用旧数据。

Batch E: 传感器端最小协议

- ☐ 实现控制帧解析和 CRC。
- ☐ 实现核心命令。
- ☐ 实现固定 44 字节快报。
- ☐ 实现退出上报。
- ☐ 通过 golden frame 对拍。

完成标准：传感器端和 Python/CPU2 golden frame 字节级一致。

Batch F: 硬件联调和认证证据

- ☐ 短线联调。
- ☐ 200 米 RS485 长线 9600 bps 联调。
- ☐ 故障注入。
- ☐ 长稳测试。
- ☐ 形成验证记录和 SIL 追踪资料。

完成标准：协议满足 9600 bps 实时性预算，异常帧不进入安全相关测量值，认证证据可追溯。

11. 关键风险和控制措施

风险	影响	控制措施
CRC32C 参数不一致	两端互不兼容	Python golden frame 先行，C 端逐帧对拍
快报偏移错误	CPU2 解析出错误密度或温度	固定偏移宏，禁止结构体强转
周期上报占满 9600 bps 链路	影响退出命令和实时性	默认 1000 ms 周期，保留 <code>control_guard_ms</code>
<code>stream_seq</code> 回绕处理错误	长稳后误判或漏判	仿真覆盖 <code>65535 -> 0</code> ，硬件长稳记录
<code>sample_counter</code> 冻结未发现	使用冻结数据	会话层冻结计数和 <code>data_age_ms</code> 双重判断
配置变化后仍按旧倍率解释	产生危险测量值	<code>safety_param_crc</code> 和 <code>config_epoch</code> 双重绑定
旧协议路径被误破坏	现场旧传感器不可用	新协议独立文件，旧路径保留，启用条件显式
旧源码编码被转码	中文注释或字符串乱码	改旧 <code>.c/.h</code> 前确认编码，避免全文件重写
认证资料滞后	实现完成后无法追溯	每批次同步 golden frame、测试记录和评审结论

12. 最小可交付定义

第一轮可交付必须同时满足：

- CPU2 能通过新协议完成 HELLO、SET_MEASURE_MODE(DENSITY)、READ_MEAS_COMBO。
- CPU2 能进入 PERIODIC_UPLINK，接收连续 REPORT_MEAS_FAST，并正常退出后继续双向通信。
- 快报固定 44 字节，9600 bps 下 1000 ms 周期稳定。
- CRC 错、长度错、地址错、旧会话、旧流、config_epoch 错、数据过期均不会更新安全相关测量值。
- 旧 DSM/LTD 协议路径仍可回退。
- golden frame、故障注入记录、长线测试记录和标准追踪矩阵同步归档。